

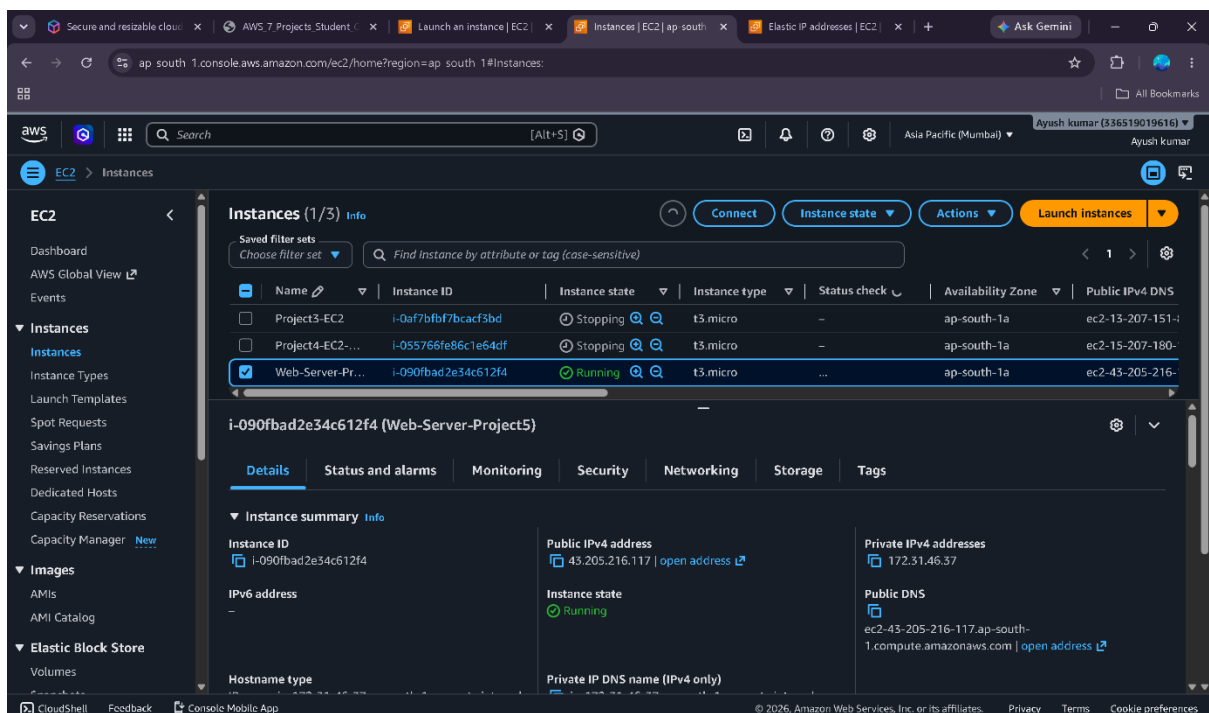
Project 5: EC2 Live Website + Database Integration

Objective

To host a full-stack PHP website on an AWS EC2 instance that dynamically reads and displays data from an Amazon RDS MySQL database.

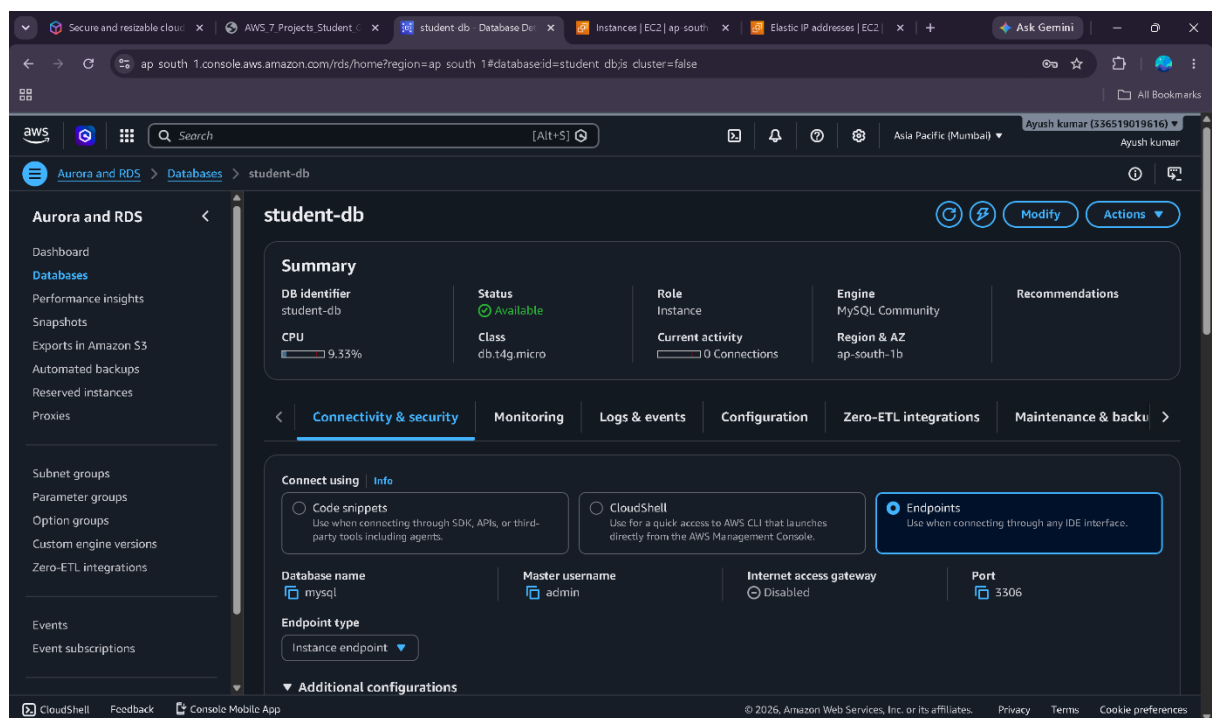
Step 1: Launch an AWS EC2 Instance

1. Open the **AWS Management Console** and navigate to the **EC2 Dashboard**.
2. Click on **Launch instance**.
3. **Name:** Enter Web-Server-Project5.
4. **Application and OS Images (AMI):** Select **Amazon Linux 2023** (Free Tier eligible).
5. **Instance type:** Choose t2.micro (or t3.micro depending on region availability).
6. **Key pair:** Select an existing key pair or create a new one to access the instance via SSH.
7. **Network settings:** * Create a new Security Group.
 - Allow **SSH (Port 22)**, **HTTP (Port 80)**, and **HTTPS (Port 443)** from anywhere (0.0.0.0/0).
8. Click **Launch instance**.



Step 2: Configure the RDS Database

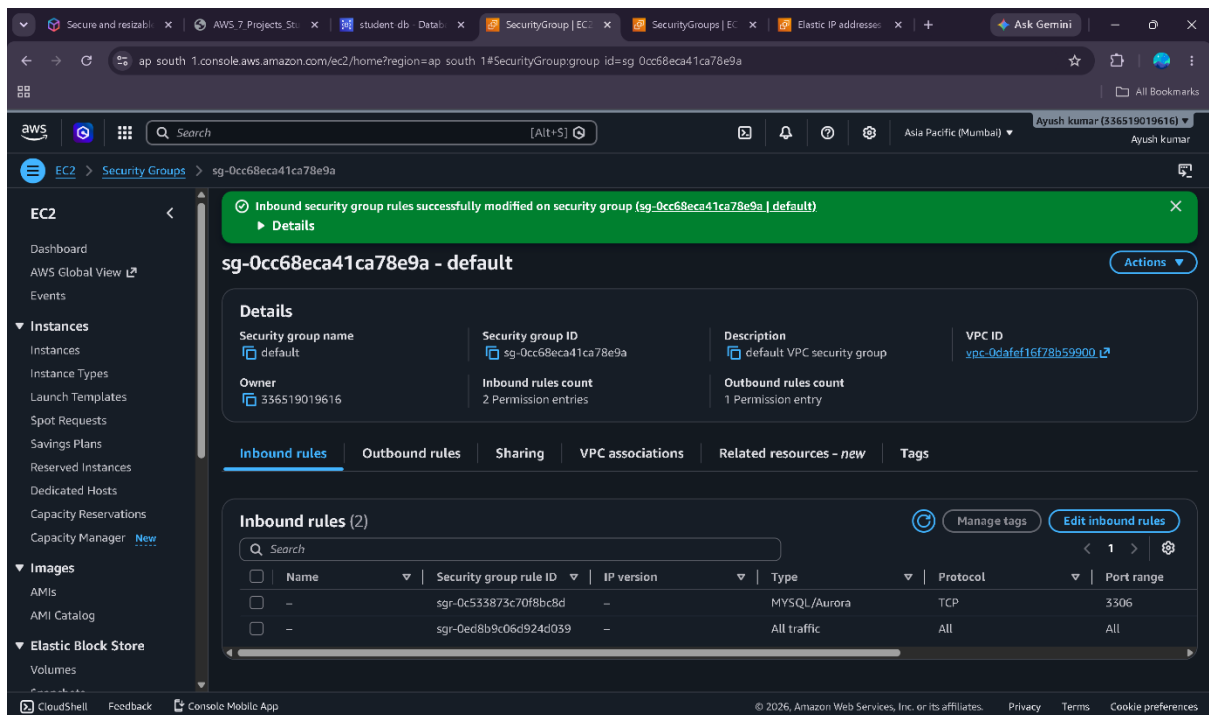
1. Navigate to the **RDS Dashboard** and click **Create database**.
2. **Engine options:** Select **MySQL**.
3. **Templates:** Choose **Free Tier**.
4. **Settings:**
 - **DB instance identifier:** student-db
 - **Master username:** admin
 - **Master password:** *[Set your secure password]*
5. **Connectivity:**
 - **VPC:** Select the same VPC where your EC2 instance is hosted.
 - **Public access:** Select **No** (Best security practice).
6. Click **Create database** and wait for the status to become *Available*.
7. Click on student-db and copy the **Endpoint** string from the *Connectivity & security* tab.



Step 3: Update Security Group for Database Connectivity

To allow your EC2 web server to talk to your RDS database, you must update the database's security rules.

1. Go to the **RDS Dashboard**, select your database, and look under the **Security** section to find its Security Group.
2. Edit the **Inbound rules** of the RDS Security Group.
3. Add a new rule:
 - **Type:** MySQL/Aurora (Port 3306)
 - **Source:** Select the **Security Group ID of your EC2 instance** (instead of Anywhere).
4. Save the rules.



Step 4: Install Apache Web Server and PHP on EC2

1. Connect to your EC2 instance using SSH or **EC2 Instance Connect**.
2. Run the following commands sequentially to update packages, install Apache, PHP, and the MySQL extension for PHP:

Bash

```
# Update the package index
```

```
sudo yum update -y
```

Install Apache web server and PHP modules

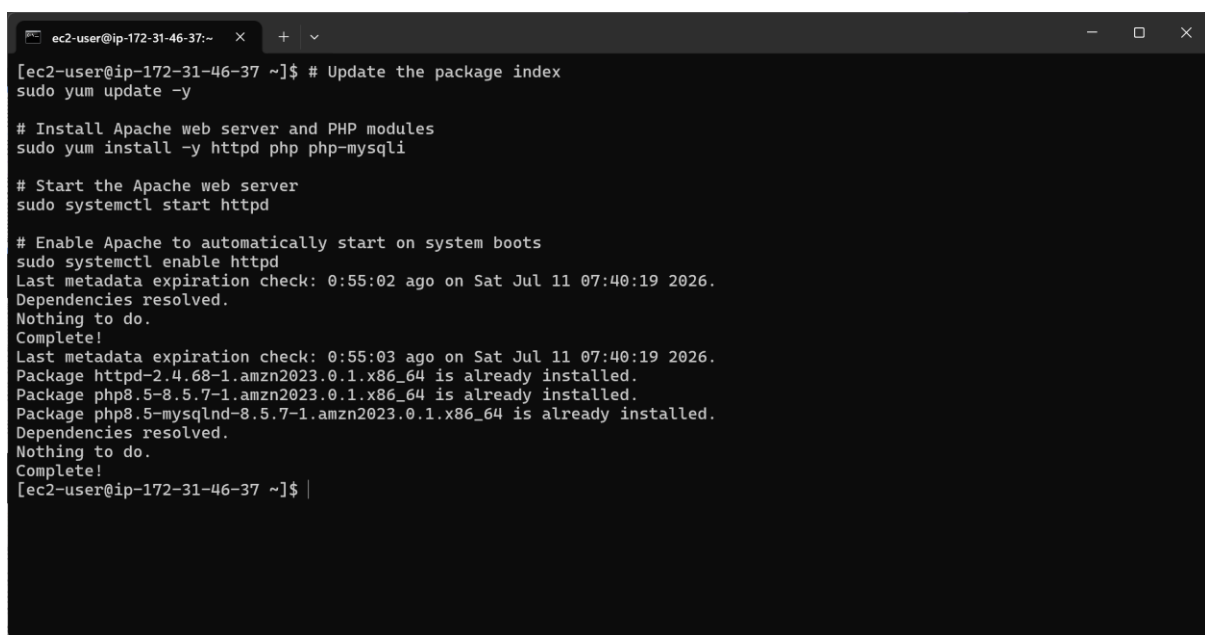
```
sudo yum install -y httpd php php-mysql
```

Start the Apache web server

```
sudo systemctl start httpd
```

Enable Apache to automatically start on system boots

```
sudo systemctl enable httpd
```



```
ec2-user@ip-172-31-46-37:~$ # Update the package index
sudo yum update -y

# Install Apache web server and PHP modules
sudo yum install -y httpd php php-mysql

# Start the Apache web server
sudo systemctl start httpd

# Enable Apache to automatically start on system boots
sudo systemctl enable httpd
Last metadata expiration check: 0:55:02 ago on Sat Jul 11 07:40:19 2026.
Dependencies resolved.
Nothing to do.
Complete!
Last metadata expiration check: 0:55:03 ago on Sat Jul 11 07:40:19 2026.
Package httpd-2.4.68-1.amzn2023.0.1.x86_64 is already installed.
Package php8.5-8.5.7-1.amzn2023.0.1.x86_64 is already installed.
Package php8.5-mysqlnd-8.5.7-1.amzn2023.0.1.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-172-31-46-37 ~]$
```

Step 5: Deploy the Web Application Code

1. Navigate to the default Apache web directory:

Bash

```
cd /var/www/html
```

2. Create and edit a new file named index.php:

Bash

```
sudo nano index.php
```

3. Paste the following PHP script into the editor. *(Make sure to replace <rds-endpoint> and yourpassword with your actual database endpoint and password):*

PHP

```

<?php
$servername = "<rds-endpoint>";
$username = "admin";
$password = "yourpassword";
$dbname = "studentdb";

// Create database connection
$conn = new mysqli($servername, $username, $password, $dbname);

// Check connection stability
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

echo "<h1>Connected successfully to the RDS Database!</h1>";
echo "<h3>Student Records:</h3>";

// Fetch and display data from the student table
$result = $conn->query("SELECT * FROM students");

if ($result && $result->num_rows > 0) {
    while($row = $result->fetch_assoc()) {
        echo "ID: " . $row["id"] . " | Name: " . $row["name"] . "<br>";
    }
} else {
    echo "No records found or table does not exist.";
}

$conn->close();
?>

```

4. Save the file (Ctrl+O, then Enter) and exit the editor (Ctrl+X).
5. Change ownership permissions so Apache can serve the file properly:

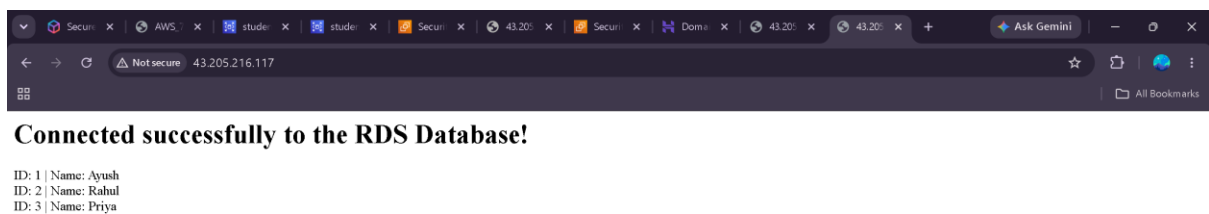
Bash

```
sudo chown -R apache:apache /var/www/html
```

```
sudo systemctl restart httpd
```

Step 6: Live Verification

1. Go back to the **EC2 Dashboard** and copy the **Public IPv4 address** of your web server instance.
2. Open a new tab in your web browser and navigate to: `http://<ec2-public-ip>`
3. The page should load cleanly, displaying **"Connected successfully to the RDS Database!"** along with the list of student names directly fetched from your cloud database.



Conclusion

The successful deployment of Project 5 effectively demonstrates the practical integration of core cloud computing services using **Amazon Web Services (AWS)**. By launching an **EC2 instance** to handle web application delivery and provisioning an isolated **Amazon RDS MySQL database**, we achieved a robust, full-stack operational architecture.